

Raft vs. PBFT: An Experimental Analysis of Consensus Protocols Under Crash and Byzantine Fault Models

Ilyas Bakchiche¹ and Arthur Bohin²

¹Department of Computer Science, UPC FIB,
ilyas.bakchiche@estudiantat.upc.edu

²Department of Computer Science, UPC FIB, arthur.bohin@estudiantat.upc.edu

June 2026

Abstract

This paper presents a comparative study of two major families of distributed consensus: RAFT, designed for crash fault tolerance (CFT), and PBFT, designed for Byzantine fault tolerance (BFT). The goal is not merely to measure which protocol is faster in absolute terms, but to explain why their behaviors diverge depending on the fault model, the number of nodes, network latency, and the presence of slow or malicious participants. Experimental results demonstrate that RAFT achieves very low latency and high stability when faults remain non-Byzantine. PBFT, by contrast, incurs higher communication overhead but addresses a strictly stronger threat model: it preserves safety even when a subset of participants behaves arbitrarily. This difference illustrates a fundamental trade-off in distributed systems — the more adversarial the fault model, the more a protocol must exchange, verify, and cross-check information before committing.

Keywords: distributed consensus, Raft, PBFT, crash fault tolerance, Byzantine fault tolerance, performance evaluation, fault injection.

Contents

1	Introduction	3
2	Theoretical Background	3
2.1	The Consensus Problem	3
2.2	Raft: CFT Leader-Based Consensus	3
2.3	PBFT: BFT Consensus via Cross-Quorum Verification	4
3	Source Code and Repository	5
4	Experimental Architecture	5
4.1	System Components	5
4.2	Measured Metrics	6

5	Experimental Scenarios	6
5.1	Scenario 1: Asymmetric Delay on Leader or Primary	6
5.2	Scenario 2: Scalability with Cluster Size	7
5.3	Scenario 3: Slow-Node Sensitivity	8
5.4	Scenario 4: Message Complexity	8
6	Quantitative Summary	9
7	Scientific Discussion	9
7.1	Why Raft Outperforms PBFT Under Benign Conditions	9
7.2	Why PBFT Incurs Higher Cost	10
7.3	Protocol Selection by Threat Model	10
7.4	Connection to the CAP Theorem	10
8	Limitations	10
9	Conclusion	11
A	Raw Experimental Data	12
A.1	Asymmetric Delay Scenario	12
A.2	Scalability Scenario	12

1. Introduction

Consensus is the mechanism that allows multiple machines to agree on a common sequence of operations despite delays, crashes, and network uncertainty. It underpins many critical systems: replicated databases, permissioned blockchains, financial ledgers, cloud infrastructure, industrial control systems, and coordination platforms.

Two major fault hypotheses dominate the literature. In the *Crash Fault Tolerant* (CFT) model, a node may stop, slow down, or lose messages, but it does not intentionally lie. RAFT belongs to this family [1]. In the *Byzantine Fault Tolerant* (BFT) model, a node may exhibit arbitrary behavior: sending contradictory values, forging messages, or colluding with an attacker. PBFT belongs to this second family [2].

The central scientific question examined in this paper is:

Under what conditions are RAFT and PBFT most appropriate, and what cost must be paid to move from a crash-tolerant consensus to one tolerant of Byzantine behavior?

The paper is grounded in an experimental implementation of both protocols, augmented by a chaos engine capable of injecting delays, crashes, restarts, partitions, and Byzantine behaviors. All source code is publicly available (see Section 3).

The remainder of this paper is organized as follows. Section 2 reviews the theoretical foundations of both protocols. Section 3 describes the public repository. Section 4 details the experimental architecture. Section 5 presents and interprets the four benchmark scenarios. Section 6 summarizes quantitative results. Section 7 discusses the findings in depth. Section 8 acknowledges limitations. Section 9 concludes.

2. Theoretical Background

2.1. The Consensus Problem

Definition 2.1 (Consensus). A consensus protocol operating over n processes must satisfy three fundamental properties:

- (i) **Agreement**: no two correct processes decide on different values.
- (ii) **Validity**: any decided value must have been proposed by some process.
- (iii) **Liveness**: if the system eventually stabilizes, all correct processes eventually decide.

In a purely asynchronous system, the FLP impossibility result [3] proves that no deterministic protocol can simultaneously guarantee termination and tolerate a single process failure. Practical protocols therefore introduce realistic assumptions: eventually bounded message delays, temporary leaders, or randomized sub-protocols.

2.2. Raft: CFT Leader-Based Consensus

RAFT [1] is a consensus protocol designed above all for understandability and implementability. It rests on three sub-problems:

- **Leader election**. A node becomes a candidate if it stops receiving heartbeats within a configurable election timeout.

- **Log replication.** The elected leader appends client requests to its log and replicates them to followers via `AppendEntries` RPCs.
- **Majority commit.** An entry is considered committed once durably stored on a majority of nodes.

Proposition 2.1. *With n nodes, RAFT tolerates up to*

$$f_{\text{crash}} = \left\lfloor \frac{n-1}{2} \right\rfloor \quad (1)$$

simultaneous crash faults.

This efficiency stems from a strong implicit assumption: nodes do not forge or tamper with messages. A compromised leader can therefore commit incorrect data if Byzantine faults are possible.

2.3. PBFT: BFT Consensus via Cross-Quorum Verification

PBFT [2] targets a more severe threat model. The protocol tolerates up to f Byzantine nodes provided that:

$$n \geq 3f + 1 \quad (2)$$

Its normal-case operation follows three phases:

1. **Pre-prepare.** The primary broadcasts the request with its digest and sequence number.
2. **Prepare.** Each replica broadcasts a `PREPARE` message to confirm it has observed the same proposal.
3. **Commit.** Once $2f + 1$ consistent `PREPARE` messages are collected, replicas broadcast `COMMIT`; a replica executes the request upon receiving $2f + 1$ `COMMIT` messages.

Remark 2.1. *Safety derives from quorum intersection: any two sets of $2f + 1$ replicas share at least one correct replica, preventing two contradictory values from being simultaneously committed across views.*

The price is a quadratic per-round communication complexity of $O(n^2)$, since replicas must exchange messages among themselves in both the `Prepare` and `Commit` phases.

3. Source Code and Repository

GitHub Repository

https://github.com/vyynq/DS_project.git

The repository contains all components described in this paper:

- **raft/** — RAFT node implementation (follower, candidate, leader states; log replication; metrics collection)
- **pbft/** — PBFT node implementation (normal mode, view-change, Byzantine states; prepare/commit log management)
- **chaos/** — Chaos Engine for controlled fault injection (crashes, latency spikes, partitions, Byzantine nodes)
- **benchmarks/** — Scenario runners exporting results to CSV/JSON
- **visualizer/** — WebSocket-based real-time GUI for cluster state visualization
- **demo.py** — Demonstration script with slowed timings for presentations
- **README.md** — Setup instructions and scenario documentation

Language: Python 3.11 (asyncio)

4. Experimental Architecture

4.1. System Components

The experimental framework is organized around four modules:

- **Protocol nodes.** RAFT nodes implement follower, candidate, and leader state machines. PBFT nodes implement the three-phase normal-case protocol, view-change logic, and Byzantine state simulation.
- **Chaos Engine.** Controlled injection of crashes, latency spikes, asymmetric delays, network partitions, and Byzantine behaviors.
- **Client driver.** Submits a configurable request stream and tracks submission timestamps for end-to-end latency measurement.
- **Benchmarking layer.** Exports raw results to structured CSV/JSON for reproducible analysis.

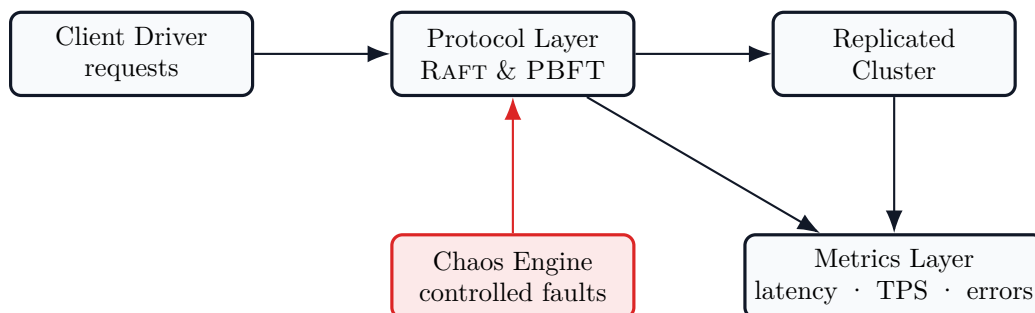


Figure 1: Experimental architecture used to compare both protocols.

4.2. Measured Metrics

Five primary metrics are collected (Figure 1):

- **Throughput** — committed requests per second (req/s).
- **Mean latency** — average time from client submission to commit (ms).
- **p_{95} latency** — 95th-percentile commit latency (ms).
- **Error rate** — fraction of requests not committed within timeout (%).
- **Message complexity** — estimated messages exchanged per round.

5. Experimental Scenarios

Note on scope. Benchmarks are executed within a local simulation. Results reveal protocol-level trends but do not replace evaluation on a physically distributed infrastructure with separate clocks and network stacks. Limitations are discussed in Section 8.

5.1. Scenario 1: Asymmetric Delay on Leader or Primary

An artificial delay is injected exclusively on the RAFT leader or the PBFT primary to observe each protocol’s sensitivity to a central-node bottleneck.

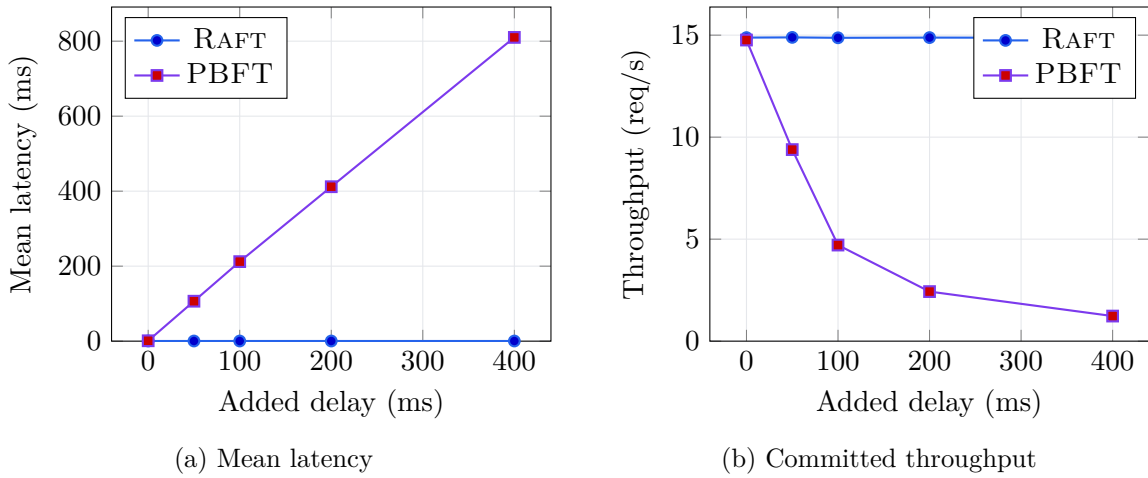


Figure 2: Impact of a delay applied to the leader or primary (Section 5.1).

RAFT remains essentially constant across all injected delays: because majority acknowledgment does not depend on leader-to-leader coordination, the bottleneck is masked. PBFT exhibits near-linear latency growth: at 400 ms of injected delay, mean latency reaches 810 ms, reflecting the multiplicative effect of delays propagating across the Pre-prepare, Prepare, and Commit message exchanges before a commit can be finalized.

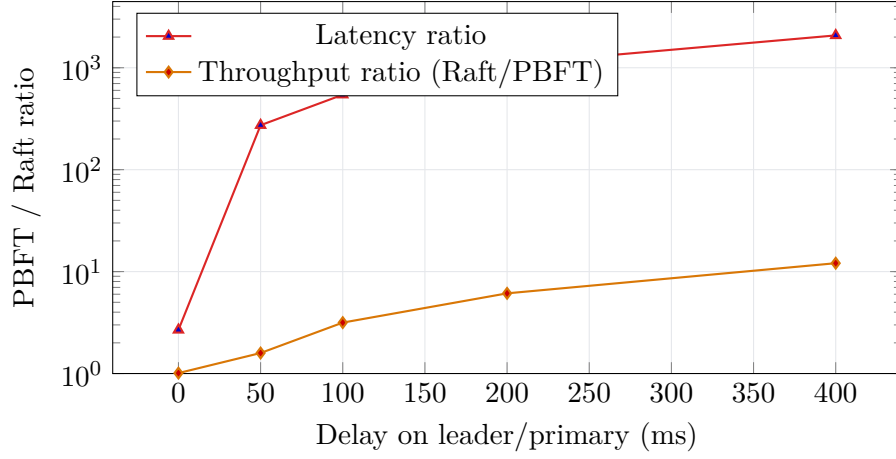
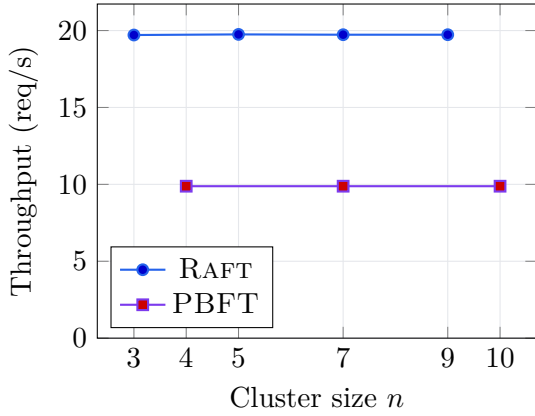


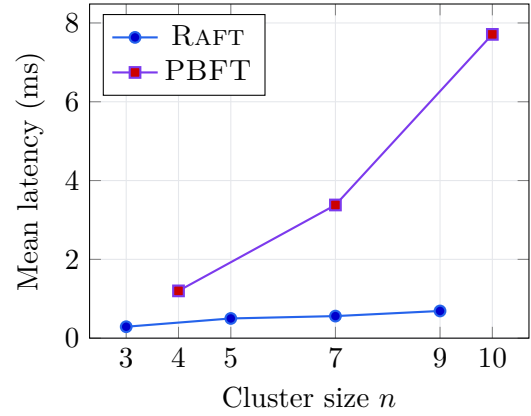
Figure 3: Relative degradation under asymmetric delay (log scale). The growing gap confirms the multi-phase sensitivity of PBFT.

5.2. Scenario 2: Scalability with Cluster Size

RAFT is tested with $n \in \{3, 5, 7, 9\}$ nodes; PBFT with $n \in \{4, 7, 10\}$ nodes, corresponding to $f \in \{1, 2, 3\}$ tolerated Byzantine faults respectively (see Equation (2)).



(a) Throughput vs. cluster size



(b) Mean latency vs. cluster size

Figure 4: Scalability results across benchmark configurations (Section 5.2).

RAFT throughput remains stable near 19.7 req/s with only a slight latency increase from 0.29 ms ($n = 3$) to 0.69 ms ($n = 9$). PBFT latency grows more sharply, from 1.20 ms at $n = 4$ to 7.71 ms at $n = 10$, consistent with the $O(n^2)$ all-to-all communication pattern.

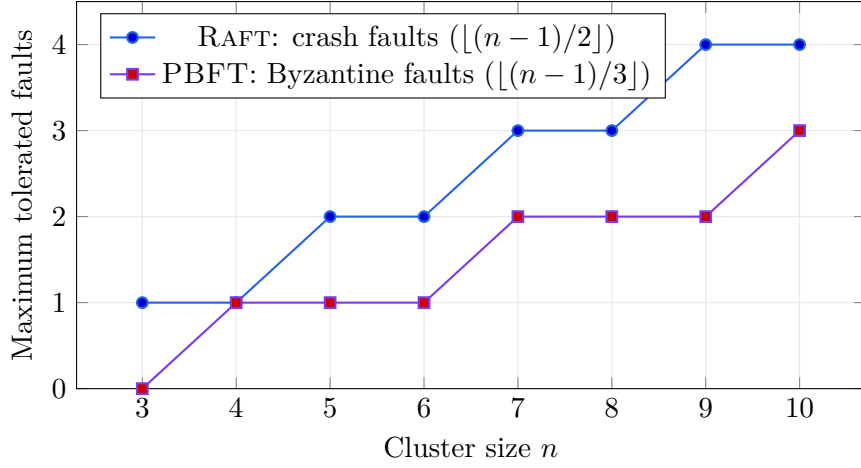


Figure 5: Theoretical fault tolerance capacity as a function of n . RAFT tolerates more faults per node, but PBFT defends against a strictly stronger class of failure.

5.3. Scenario 3: Slow-Node Sensitivity

A delay of 500 ms is injected on a single node. Two configurations are evaluated: (a) the slow node is a follower/replica; (b) it is the leader/primary.

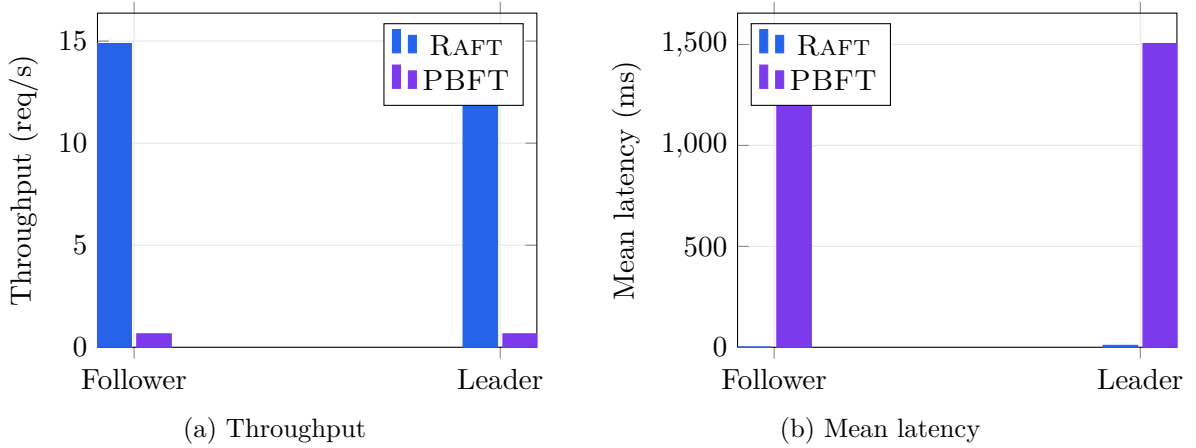


Figure 6: Impact of a single slow node with 500 ms injected delay (Section 5.3).

RAFT tolerates a slow follower well: the leader can satisfy the majority quorum without waiting for every node. When the leader itself is slow, throughput drops from 14.88 to 12.31 req/s and the error rate rises to 8.33%. PBFT is substantially more sensitive: in both configurations, throughput collapses to 0.66 req/s and mean latency exceeds 1.5 s, because the Prepare/Commit phases require acknowledgment from $2f + 1$ replicas, making the protocol as slow as its slowest required responder.

5.4. Scenario 4: Message Complexity

Theoretical message complexity provides the analytical explanation for all preceding observations. For a single consensus round, RAFT requires approximately $2n$ messages (leader-to-follower broadcast plus acknowledgments), whereas PBFT requires approximately $3n^2$ messages (broadcast + all-to-all Prepare + all-to-all Commit).

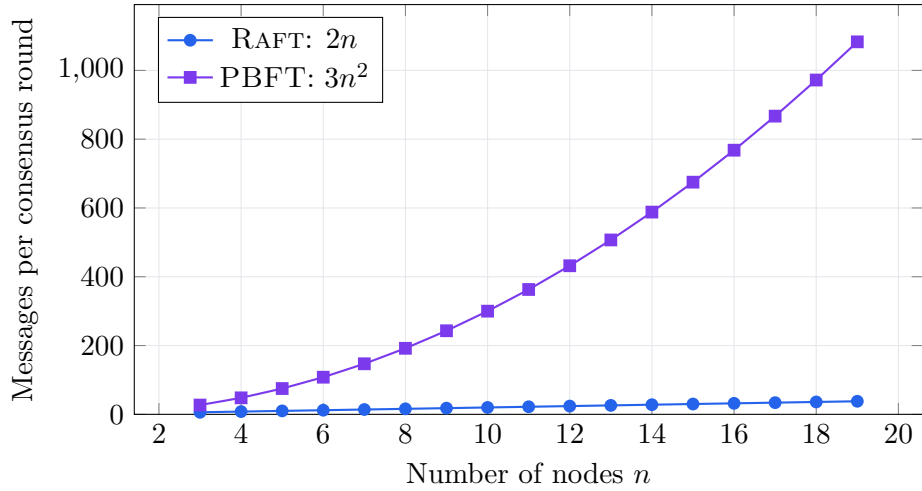


Figure 7: Theoretical message complexity per consensus round. The quadratic growth of PBFT is the direct cause of its latency sensitivity in Scenarios 5.1–5.3.

6. Quantitative Summary

Table 1 consolidates the most significant experimental results across all four scenarios.

Table 1: Summary of experimental results across benchmark scenarios.

Scenario	Protocol	Throughput (req/s)	Mean lat. (ms)	Error (%)
Delay 0 ms on primary/leader	RAFT	14.88	0.38	0.00
Delay 0 ms on primary/leader	PBFT	14.76	1.02	0.00
Delay 400 ms on primary/leader	RAFT	14.87	0.39	0.00
Delay 400 ms on primary/leader	PBFT	1.23	810.04	0.00
Scalability, $n = 7$	RAFT	19.73	0.56	0.00
Scalability, $n = 7$	PBFT	9.88	3.38	0.00
Slow follower/replica	RAFT	14.88	0.40	0.00
Slow follower/replica	PBFT	0.66	1504.43	0.00
Slow leader/primary	RAFT	12.31	9.55	8.33
Slow leader/primary	PBFT	0.66	1504.74	0.00

7. Scientific Discussion

7.1. Why Raft Outperforms PBFT Under Benign Conditions

RAFT minimizes the number of message exchanges required to reach a decision. The leader imposes entry ordering, followers acknowledge, and a simple majority suffices. This architecture accounts for:

- Sub-millisecond latency under normal operation;
- Strong resilience to slow followers, since only a majority quorum is required;
- Practical scalability for small and medium cluster sizes;
- Fast recovery after leader failures when election timeouts are well-calibrated.

The fundamental limitation is the implicit trust placed in the leader and in message integrity. If the leader is compromised, RAFT provides no native mechanism to detect tampered values.

7.2. Why PBFT Incurs Higher Cost

PBFT does not merely accept the primary’s proposal. Replicas must independently verify that they observe the same proposal and then confirm that a consistent quorum is ready to commit. This produces several compounding overheads:

- $O(n^2)$ messages per request rather than $O(n)$;
- Delays propagating multiplicatively across three protocol phases;
- Additional memory to maintain per-view prepare and commit logs;
- Significantly more complex view-change logic when the primary fails.

Crucially, this cost purchases a property that RAFT structurally cannot provide: safety against malicious, inconsistent, or equivocating participants.

7.3. Protocol Selection by Threat Model

Table 2 synthesizes the qualitative trade-offs. The choice between RAFT and PBFT should be driven first by threat modeling, not by performance benchmarking.

Table 2: Qualitative comparison of RAFT and PBFT.

Criterion	Raft	PBFT
Fault model	Crash, slowdown, message loss	Crash, slowdown, arbitrary messages, malicious nodes
Quorum requirement	Simple majority ($\lfloor (n-1)/2 \rfloor$)	BFT quorum ($n \geq 3f + 1$)
Message complexity	Linear — approx. $2n$	Quadratic — approx. $3n^2$
Normal-case latency	Very low (< 1 ms in simulation)	Higher due to multi-phase rounds
Leader sensitivity	High latency/errors if leader is slow; no protection against malicious leader	High latency at slow primary, but inconsistent values are detected
Natural use cases	Replicated databases, cloud orchestration, administratively trusted systems	Permissioned blockchains, multi-operator consortia, adversarial environments

7.4. Connection to the CAP Theorem

The CAP theorem [4] asserts that a distributed system cannot simultaneously guarantee consistency, availability, and partition tolerance. Both RAFT and PBFT prioritize consistency over availability: under a partition that isolates the majority quorum, RAFT stalls rather than committing from an isolated minority. PBFT adds a further constraint: safety must be preserved even when up to f of the n participants are actively adversarial, which tightens the quorum requirement and thereby reduces availability margins.

8. Limitations

This study is intentionally experimental and pedagogical. Several limitations must be noted before generalizing its conclusions:

1. Benchmarks are executed within a single-host simulation; no physical network latency, clock skew, or OS scheduling interference is present.
2. The PBFT implementation is simplified relative to a full production system; in particular, view-change is partially exercised.
3. Cryptographic signatures are simulated rather than computed, removing their overhead from latency measurements.
4. Results are sensitive to the chosen timeout parameters, cluster sizes, and load model; different calibrations would shift absolute values.
5. Modern optimizations such as request batching, threshold signatures, and linear-communication variants (HotStuff [5], Tendermint) are not evaluated.

These constraints do not invalidate the main comparative findings: the trends in Figures 2, 4, 6 and 7 are driven by structural protocol properties (message complexity, quorum size) that hold regardless of implementation substrate.

9. Conclusion

The experimental and theoretical results of this study converge on a clear picture. Under benign fault conditions — crashes, restarts, or network delays — RAFT delivers outstanding performance: sub-millisecond latency, stable throughput across cluster sizes, and strong resilience to slow followers. These properties explain why RAFT and its variants (Multi-Paxos, Zab) power most modern replicated databases and orchestration systems.

PBFT occupies a different design point. Its higher latency and quadratic message complexity are not engineering failures but the direct and necessary consequence of verifying consistency in a system where participants may lie, collude, or behave unpredictably. In adversarial environments — permissioned blockchains, financial consortia, critical infrastructure shared across independent operators — this communication overhead is not a drawback but the price of verifiable safety.

The central lesson is therefore not that one protocol dominates the other. RAFT and PBFT optimize for two genuinely different problems. RAFT is the right tool when trust is administrative and faults are accidental; PBFT is the right tool when trust must be proven and faults may be intentional. Any engineering decision between the two should begin not with a benchmark but with an honest assessment of the threat model, the trust boundaries, and the required safety guarantees.

Looking further, this binary choice between CFT and BFT has inspired a generation of hybrid protocols — HotStuff [5], Tendermint, Casper — that reduce the overhead of BFT consensus through linear communication, pipelining, and aggregated signatures. The trade-offs exposed in this paper remain the analytical foundation for evaluating all of them.

References

- [1] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (ATC'14)*, pages 305–320. USENIX Association, 2014.
- [2] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI'99)*, pages 173–186. USENIX Association, 1999.

- [3] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.
- [4] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.
- [5] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. HotStuff: BFT consensus with linearity and responsiveness. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC’19)*, pages 347–356. ACM, 2019.

A. Raw Experimental Data

A.1. Asymmetric Delay Scenario

Table 3: Full data for Scenario 1 (asymmetric delay on leader/primary).

Protocol	Delay (ms)	Throughput (req/s)	Mean lat. (ms)	p_{95} (ms)
RAFT	0	14.88	0.38	0.61
PBFT	0	14.76	1.02	1.34
RAFT	50	14.89	0.39	0.44
PBFT	50	9.39	106.52	106.89
RAFT	100	14.87	0.39	0.46
PBFT	100	4.71	212.10	212.90
RAFT	200	14.88	0.38	0.47
PBFT	200	2.43	411.84	413.38
RAFT	400	14.87	0.39	0.68
PBFT	400	1.23	810.04	813.14

A.2. Scalability Scenario

Table 4: Full data for Scenario 2 (scalability with cluster size).

Protocol	n	Tolerated faults	Throughput (req/s)	Mean lat. (ms)
RAFT	3	1 crash	19.71	0.29
RAFT	5	2 crash	19.75	0.50
RAFT	7	3 crash	19.73	0.56
RAFT	9	4 crash	19.73	0.69
PBFT	4	1 Byzantine	9.88	1.20
PBFT	7	2 Byzantine	9.88	3.38
PBFT	10	3 Byzantine	9.88	7.71